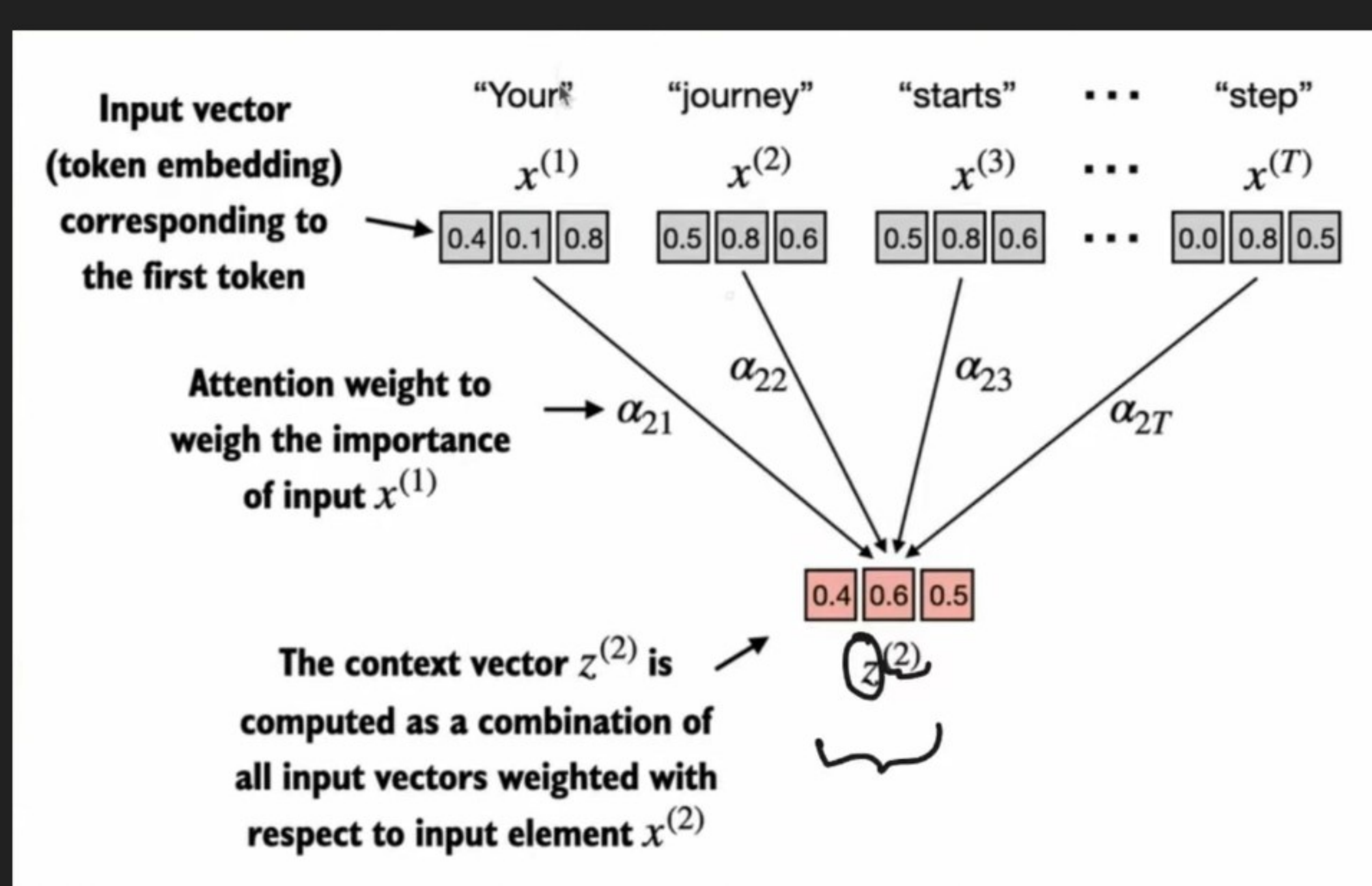
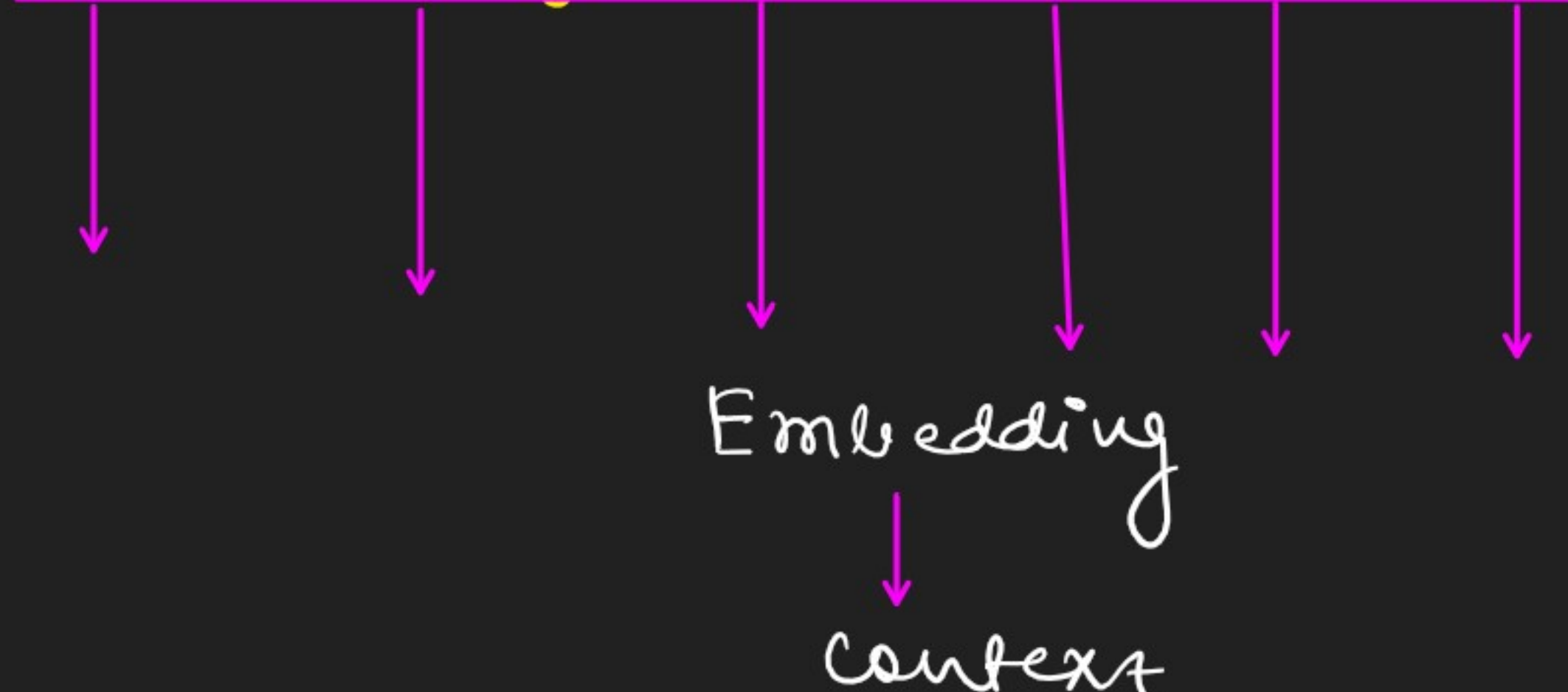


Lecture-14 (A simple self Attention

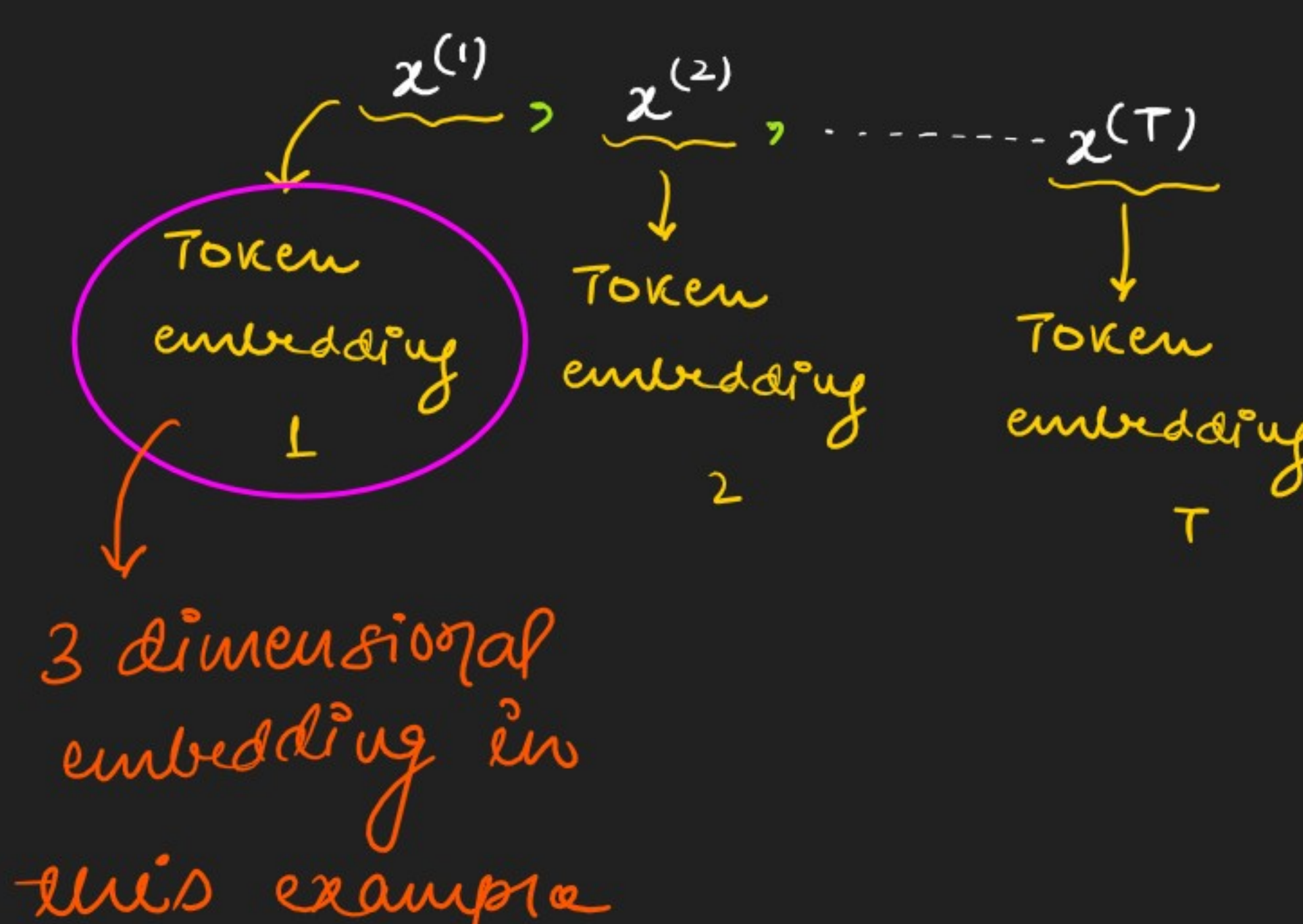
without trainable weight

(a) In this section, we will implement a simple variant of self-attention, free from any trainable weights.

eg. your journey starts with one steps



1. Input sequence: x
consist of T elements



2. Goal of self attention:

"calculate a context vector ($z^{(i)}$) for each element $x^{(i)}$ "

Context vector: Enriched embedding vector

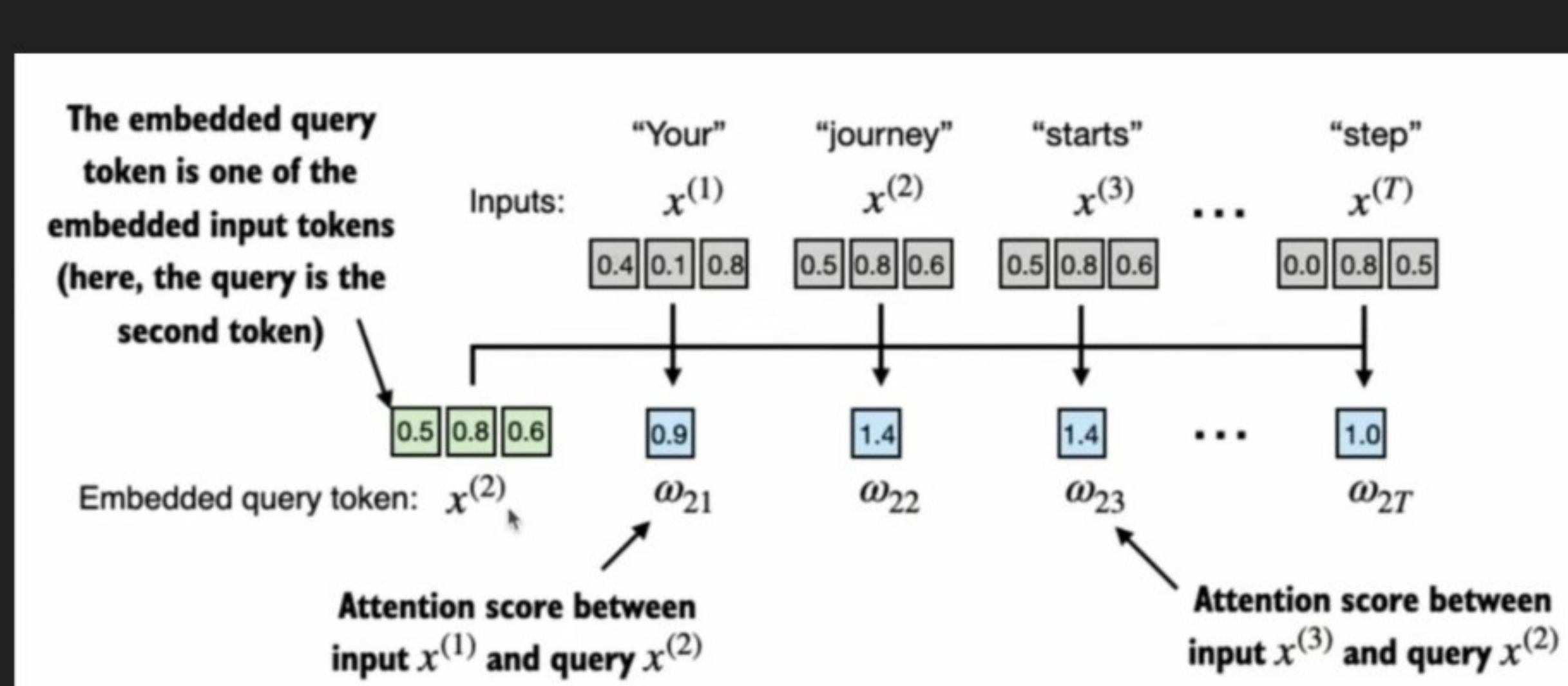
3. let's focus on 2nd element ($x^{(2)}$)

corresponding context vector is $z^{(2)}$

$z^{(2)}$ is an embedding which contain information about $x^{(2)}$ and all other input element $x^{(1)}$ to $x^{(T)}$

4. later, we will add trainable weight that help the LLM learn to construct these context vector; so that they are relevant for the LLM to generate the next token.

5. First step of implementing self-attention is to compute the intermediate values w , also referred to as attention scores.



6. Intermediate attention scores are calculated b/w the query and each input token

dot product b/w query token and every other input token.

dot product quantities how much two vector are aligned.

In the context of self-attention mechanism, dot product determines the extent to which element of a sequence attend to one another $\rightarrow$ higher the dot product, higher the similarity and attention scores b/w two elements.

7. In the next step, we normalise each of the attention scores we calculated previously.

$[x_1, x_2, x_3, x_4, x_5, x_6]$

softmax normalization
(we take exponent of every element)

$\left[\frac{e^{x_1}}{\text{sum}} \quad \frac{e^{x_2}}{\text{sum}} \quad \frac{e^{x_3}}{\text{sum}} \quad \frac{e^{x_4}}{\text{sum}} \quad \frac{e^{x_5}}{\text{sum}} \quad \frac{e^{x_6}}{\text{sum}} \right]$

$(e^{x_1} + e^{x_2} + e^{x_3} + e^{x_4} + e^{x_5} + e^{x_6})$

$[x_1, x_2, x_3, x_4, x_5, x_6]$

Pytorch
softmax

$\left[\frac{e^{x_1 - \max}}{\text{sum}} \quad \frac{e^{x_2 - \max}}{\text{sum}} \quad \frac{e^{x_3 - \max}}{\text{sum}} \quad \frac{e^{x_4 - \max}}{\text{sum}} \quad \frac{e^{x_5 - \max}}{\text{sum}} \quad \frac{e^{x_6 - \max}}{\text{sum}} \right]$

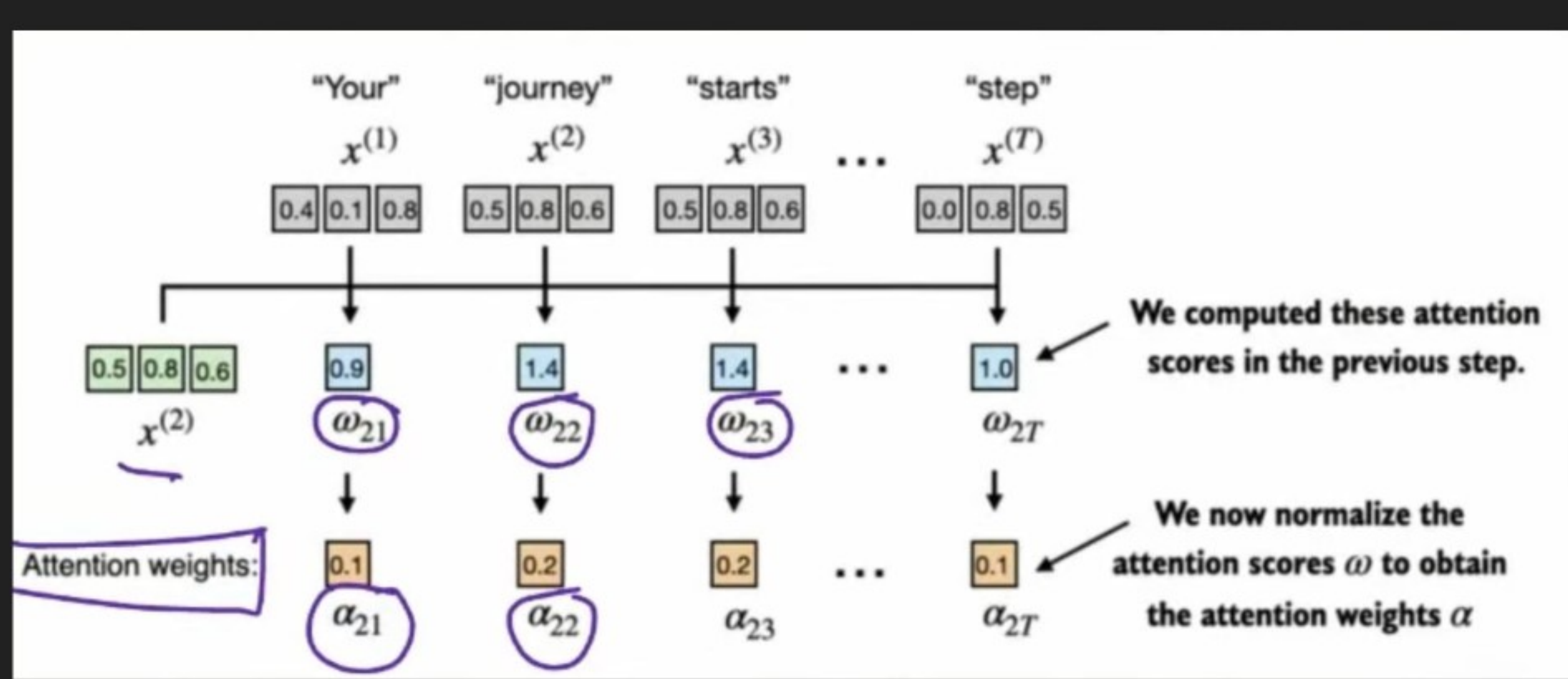
$\left(\frac{e^{x_1}}{e^{\max}} = e^{x_1 - \max} \right)$

$(e^{x_1 - \max} + e^{x_2 - \max} + \dots + e^{x_6 - \max})$

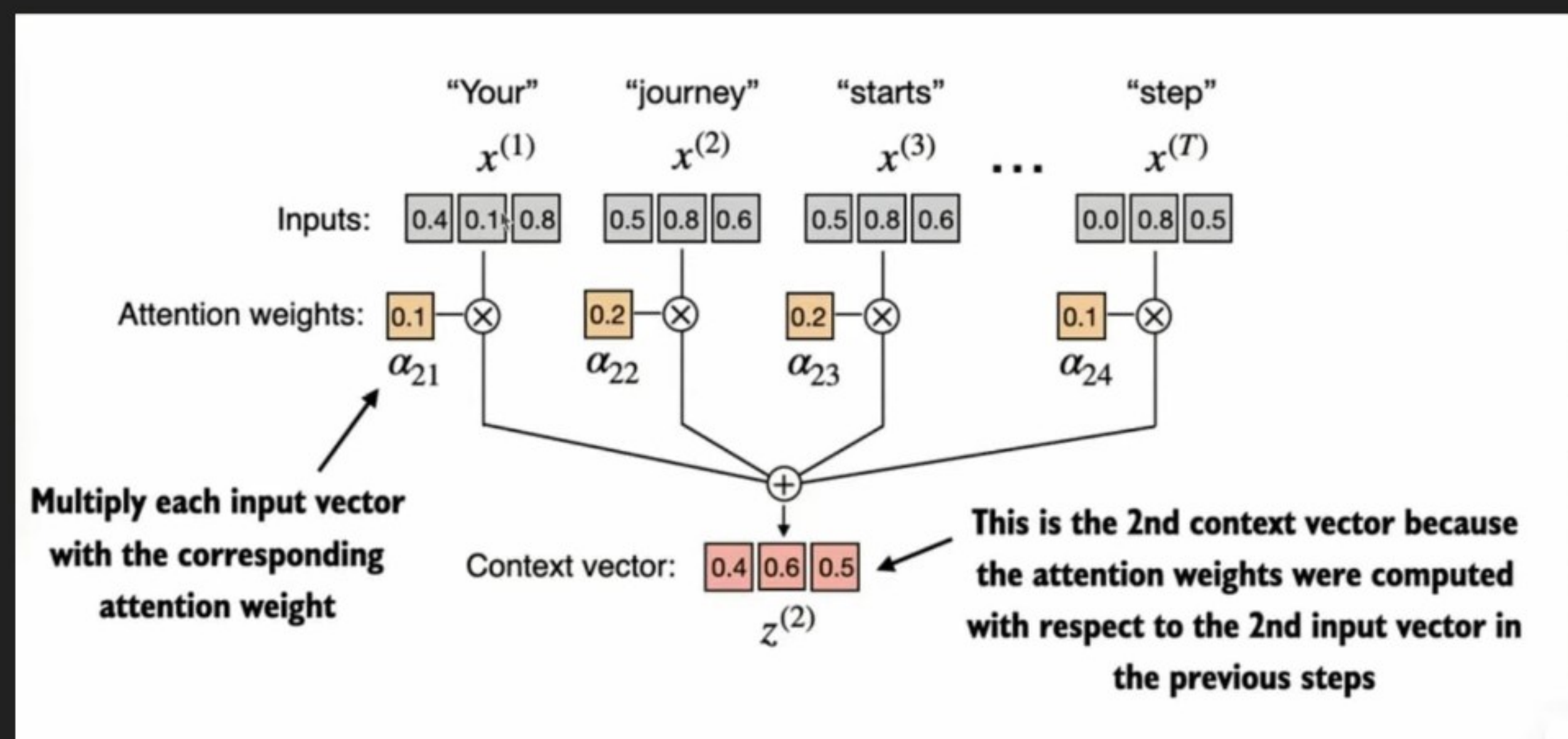
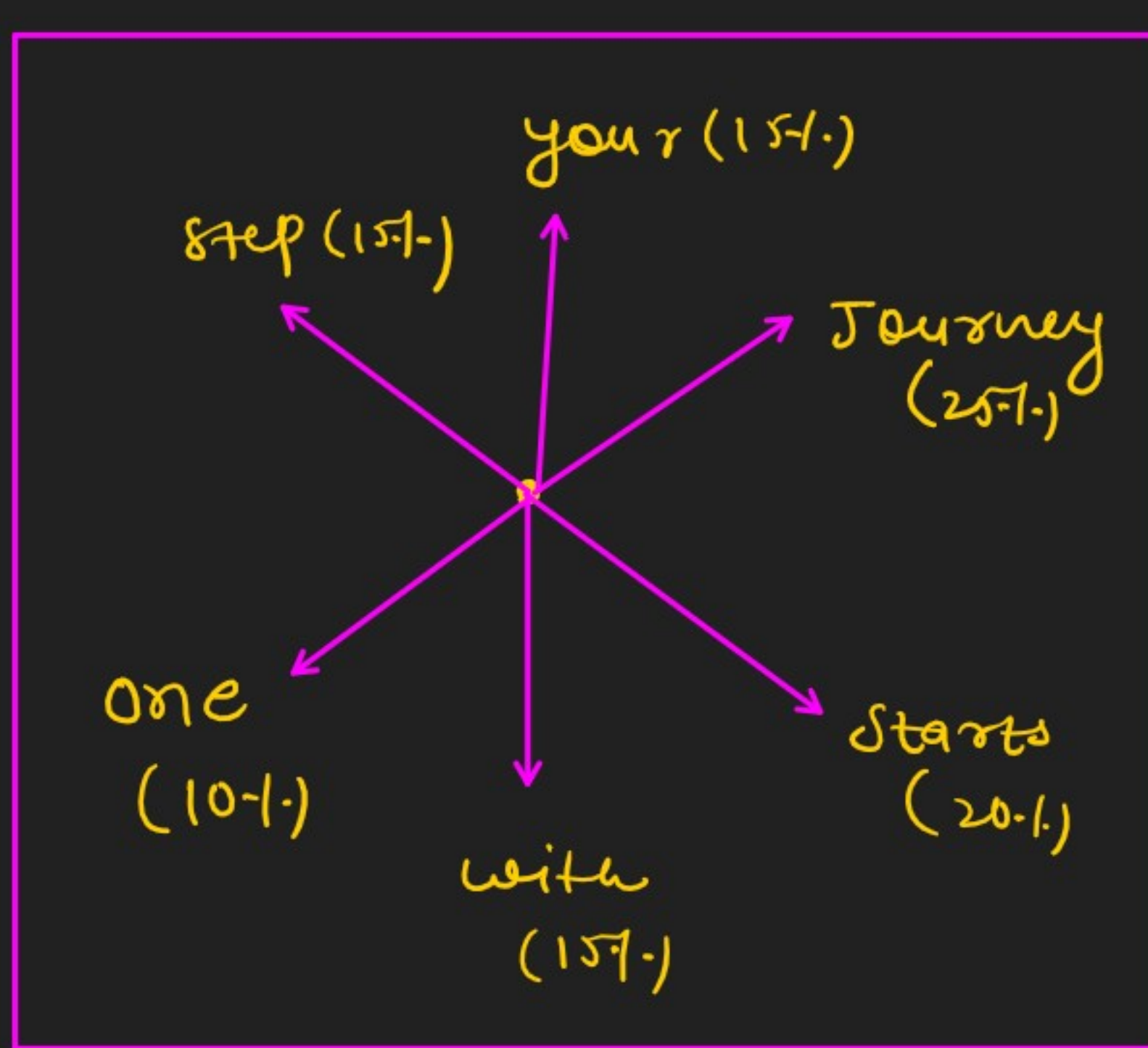
* softmax fⁿ ensure that the attention weights are always positive.

NOTE

- Naive softmax implementation (softmax-naive) may encounter numerical instability problems, such as overflow and underflow, when dealing with large or small input values.
- Therefore, in practice, it's advice to use the pytorch implementation of softmax, which has been extensively optimised for performance:



8. After, computing the normalised attention weights, we calculate the context vector $\{z^{(2)}\}$ by multiplying the embedded input token's $x^{(i)}$, with the corresponding attention weights and the summing the



9. computing attention weight for all input tokens.

we can extend similar computation as before to calculate attention weight and context vector for all inputs

	Your	journey	starts	with	one	step
*Your	0.20	0.20	0.19	0.12	0.12	0.14
*journey	0.13	0.23	0.23	0.12	0.10	0.15
*starts	0.13	0.23	0.23	0.12	0.11	0.15
*with	0.14	0.20	0.20	0.14	0.12	0.17
one	0.15	0.19	0.19	0.13	0.18	0.12
step	0.13	0.21	0.21	0.14	0.09	0.18

attention weight matrix

10. we will follow the same 3 steps as:

1-) compute attention scores

compute the attention scores as dot product b/w the inputs

2-) compute attention weights

attention weight are a normalised version of the attention scores

3-) compute attention vectors

The context vectors are computed as a weighted sum over the inputs